Web Technologies - Quiz 2

Theory prep pack: HTML forms + GET/POST, CSS box model, flexbox, media queries, positioning

Quiz date: 19 October 2026. Built from your two lecture PDFs (HTML and CSS by Asif Shahzad).

> **HOW TO USE THIS:** Everything below is explained in simple words. Short code is included because quiz questions often show a small piece of code and ask what it does or what URL it produces.

Study plan

Time	If you have 1 hour	If you have 2 hours
0-25 min	Read Part 1 (forms) fully. Memorise the GET vs POST table.	Read Part 1 slowly. Type the sample forms and open them in a browser.
25-45 min	Read Part 2 (CSS). Focus on box model maths + flexbox properties.	Read Part 2. Try the flexbox examples in a browser (change one property at a time).
45-55 min	Do the Predict-the-output questions (3D) and True/False (3F).	Do all of Part 3 with the answers covered (short Q&As first, then predict-the-output).
55-60 min	Read the one-page cheat sheet (Part 4).	Re-read the questions you got wrong, then the cheat sheet.

What the lecture slides stress (so the quiz probably will too)

- Forms: input text/password/radio/checkbox, `label for`, select + option, hidden, textarea (rows/cols), button types (submit, reset, button), HTML5 types (url, date, email), `value`, `required`, `action`, `method`, GET vs POST.
- The slides show the URL that comes out after submitting (like `register.php?first-name=Ali&gender=Male&course=OOP&course=DB`). Expect a question asking you to write that URL.
- Box model: content, padding, border, margin, and `box-sizing: border-box`.
- Positioning: static, relative, absolute, fixed, sticky.
- Flexbox: flex-direction, justify-content, align-items, flex-wrap, flex-grow, flex-shrink, flex-basis, main/cross axis, positive/negative free space.
- Media queries for responsive design.

PART 1 - HTML FORMS and the GET / POST protocol theory

1.1 What is a form?

A **form** collects input from the user and sends it to a server. The `<form>` tag wraps all the fields. Two attributes on `<form>` matter most:

Attribute	Meaning	Example
`action`	The URL (page/script) that receives the data when the form is submitted.	`action="register.php"`
`method`	HOW the data is sent: `GET` or `POST`. If you write nothing, the default is **GET**.	`method="GET"`

```
<form action="register.php" method="GET">
    <div>
        <label>First Name</label>
        <input type="text" name="first-name"/>
    </div>
    <div>
        <label>Email</label>
        <input type="email" name="email" placeholder="Email"/>
    </div>
    <input type="submit"/>
</form>
```

EXAM TRAP: A field is sent to the server ONLY IF it has a `name` attribute. The name becomes the parameter name in the URL/body (`first-name=Ali`). No name = nothing sent.

1.2 All form controls in one table

Control	Tag / type	Key points
Text box	`<input type="text">`	Single line. `value="Ali"` pre-fills it. `placeholder` shows grey hint text that disappears when typing (it is NOT a real value).
Password	`<input type="password">`	Characters are hidden on screen. This does NOT encrypt anything.
Radio	`<input type="radio">`	Choose ONE from a group. Radios with the SAME `name` form one group.
Checkbox	`<input type="checkbox">`	Choose ZERO or MORE. Several checkboxes can share one `name`; each ticked one is sent.
Drop-down	`<select>` + `<option>`	`selected` pre-selects an option. Without `value` on option, the visible text is sent.
Text area	`<textarea>`	Multi-line. Size by `rows` and `cols`. Text goes BETWEEN the tags (no `value` attribute). Has a closing tag.
Hidden	`<input type="hidden">`	Not shown, but sent with the form (e.g. `id=23`).
Submit button	`<input type="submit">`	Sends the form. Sends its own name/value only if it has a `name`.
Reset button	`<input type="reset">`	Puts all fields back to their starting values. Sends nothing.
Plain button	`<input type="button">`	Does nothing by itself. Needs JavaScript to do something.
Email / URL / Date	`type="email"`, `"url"`, `"date"`	HTML5 types. Browser checks the format and shows a message; date gives a date picker.

1.3 The label tag and the for attribute

`<label>` is the text shown next to a field. Using `for` links the label to a field. The value of `for` must match the `id` of the input. After linking, clicking the label puts the cursor in the field (and ticks a checkbox/radio). It also helps screen

readers.

```
<label for="fn">First Name</label>
<input type="text" id="fn" name="first-name"/>
```

> **EXAM TRAP:** `for` points to the input's `id`, NOT its `name`. `id` is for linking/styling; `name` is for sending data.

1.4 Radio buttons and checkboxes (very likely in the quiz)

```
<div><label>Gender</label>
   <input type="radio" name="gender" value="Male"/> Male
   <input type="radio" name="gender" value="Female"/> Female
</div>
<div><label>Courses</label>
   <input type="checkbox" name="course" value="OOP"/> OOP
   <input type="checkbox" name="course" value="DB"/> DB
   <input type="checkbox" name="course" value="Web"/> Web
   <input type="checkbox" name="course" value="OS"/> OS
</div>
```

- **Same `name` on radios = only one can be chosen.** Different names = they act independently (you could select both).
- **No `value` attribute** -> the browser sends `on`. The slide shows exactly this: `register.php?first-name=&gender=on&course=on&course=on`. The server cannot tell which one was chosen!
- **With `value`** -> you get `gender=Male&course=OOP&course=DB`.
- **Unchecked checkbox sends nothing at all.** Only ticked ones are in the URL.
- Several ticked checkboxes with the same name appear as repeated parameters: `course=OOP&course=DB`.
- Pre-select with `checked` (e.g. `<input type="radio" checked>`).

> **EXAM TRAP:** The slide has a deliberate bug: the last checkbox (OS) was written with `value="Web"`, the same as the Web checkbox. Then the server cannot tell Web from OS. Every option should have its own unique `value`.

1.5 Select (drop-down) and textarea

```
<select name="city">
    <option>Select</option>
    <option>Lahore</option>
    <option selected="selected">Karachi</option>   <!-- pre-selected -->
    <option>Islamabad</option>
</select>

<textarea name="comments" rows="6" cols="30"></textarea>
```

- Option without `value` -> text is sent: `city=Lahore`. Option with `value="1"` -> `city=1` (slide: `...&city=1&comments=hello`).
- `selected` pre-selects an option (Karachi shows first in the slide).
- `rows` = visible height in lines, `cols` = visible width in characters.

1.6 Hidden field

```
<input type="hidden" name="city" value="Lahore"/>
<input type="hidden" name="id" value="23"/>
<input type="text" name="first-name"/>
<!-- user types Ali, URL: register.php?city=Lahore&id=23&first-name=Ali -->
```

- Used to pass extra info the user does not need to see (record id, etc.).
- Parameters appear in the order they are written in the HTML.

> **EXAM TRAP:** Hidden does NOT mean secure. Anyone can see and edit it with DevTools / View Source. Never trust it for secrets or prices.

1.7 Buttons: submit, reset, button (and using a button as data)

- `submit` sends the form. `reset` clears it back to initial values. `button` does nothing without JavaScript.
- **If the submit button has no `name`, nothing from it is sent.**
- If it has a `name`, then `name=value` is sent (default value is `Submit`).
- **Several submit buttons:** only the one that was CLICKED sends its name and value. Good for Update/Delete forms.

```
<input type="text" name="first-name"/>
<input type="submit" name="action" value="Update"/>
<input type="submit" name="action" value="Delete"/>
<!-- type Ali, click Update ->  register.php?first-name=Ali&action=Update -->
```

1.8 HTML5 input types, value, required

```
<input type="email" name="email" placeholder="Email"/>
<input type="url" name="blog_url"/>
<input type="date" name="date-of-birth" required="required"/>
```

Feature	What the browser does
`type="email"`	Message if there is no @, e.g. "Please include an '@' in the email address."
`type="url"`	Message "Please enter a URL." if the format is wrong.
`type="date"`	Shows a date picker, format mm/dd/yyyy in the slide.
`required`	Blocks submission if empty: "Please fill out this field."
`value`	The starting (default) text in the field.
`placeholder`	Grey hint only. Not a value. Not submitted if the user types nothing.

REMEMBER: This is CLIENT-SIDE validation (in the browser). A user can bypass it, so the SERVER must validate again.

1.9 UX tip from the slides: update forms

When you make an **update** form, always show the **current value** in each field (use `value="..."`, `selected`, `checked`). Never ask the user to update something without showing what it is now. **Exception: password** (never show it).

1.10 How data travels - reading the URL (query string)

With GET, the browser builds: **page ? name1=value1 & name2=value2 ...**

```
register.php ? first-name=ali & password=secret
  ^page          ^param=value     ^ & separates parameters
  ? starts the query string

Empty field still appears:  register.php?first-name=&password=secret
```

EXAM TRAP: The slide 'Submit the Form' shows `password=secret` in the address bar. That is the reason you must NOT use GET for passwords.

1.11 GET vs POST (most important theory)

Point	GET	POST
Where is data sent?	In the **URL** (query string)	In the **body** of the HTTP request (not in URL)
Visible?	Yes, in the address bar, history, server logs	Not in the URL (but still readable on the network unless HTTPS is used)
Bookmark / share link	Yes - the URL contains the data	No
Size limit	Limited (URL length, around 2000 characters in practice)	No practical limit
Cached / history	Can be cached and stays in browser history	Normally not cached
Back / refresh	Safe to repeat	Browser warns before re-sending

Point	GET	POST
Typical use	Searching, filtering, reading data (**getting**)	Login, registration, saving, uploading, changing data (**sending**)
File upload	Not possible	Possible (`enctype="multipart/form-data"`)
Default?	**Yes**, default method	Must be written explicitly

EXAM TRAP: POST is NOT automatically 'secure' or 'encrypted'. It only hides data from the URL. Real protection needs HTTPS.

MEMORY TRICK: GET = **G**et information, data in the URL, can be bookmarked. POST = **P**ut/post information, **P**rivate-looking (hidden from URL), **P**lenty of data.

1.12 Styling forms (slide code)

```
<style>
    * { font-family: "Open Sans"; }              /* every element */
    label { display: inline-block; width: 100px; } /* labels line up in a column */
    form > div { margin-bottom: 20px; }          /* gap between rows */
</style>
```

- `label` is inline by default; width has no effect on inline elements, so we set `display: inline-block` first.
- `form > div` means only divs that are DIRECT children of the form.

PART 1B - Other HTML topics (quick revision)

Page structure and tag anatomy

- Structure: `<!DOCTYPE html>`, `<html>`, `<head>` (info about the page), `<body>` (what the user sees), `<h1>`, `<p>`.
- **Start tag** `<p>`, **content**, **end tag** `</p>`. **Attribute** gives extra info: `<a href="...">`. **Self-closing/empty tag** has no body: `<br/>`, `<img/>`, `<input/>`. **Nesting rule:** close first what you opened last.
- Browser **parses** HTML and **renders** it. HTML = Hyper Text (linked text) + Markup (tags) + Language (rules). File extension `.html` (or `.htm`).
- Comment: `<!-- text -->`

Head tag, meta tags, direction

Item	Purpose
`<meta charset="UTF-8"/>`	Character encoding. Needed to show Urdu and other languages correctly.
`<meta name="description" content="..."/>`	Short summary; Google can show it under the page title in search results (SEO).
`<meta name="keywords">`, `name="author"`	Keywords and author of the page.
`<meta property="og:title"...>` (Open Graph)	Controls URL, title, description and image shown when the page is shared on social sites. Also `og:url`, `og:description`, `og:image`.
`dir="rtl"` (e.g. on `<body>`)	Text direction right-to-left (Urdu/Arabic). `ltr` is the default.
Tags allowed in head	`meta`, `title`, `style`, `link` (external CSS), `script`.

Links, images

- `<a href="url">Anchor text</a>` - `a` = anchor, `href` = hypertext reference (the target). `target="_blank"` opens in a new tab. `title` shows a tip/helps screen readers.
- **Relative URL** = link inside the same site (`about.html`). **Absolute URL** = full address (`http://comsats.edu.pk`), used for external links.
- Link to a part of the same page with a fragment identifier (`#section`) / `name` or `id`.
- `<img src="..." alt="..." width height>`. `alt` = text if image fails + screen readers + SEO. Give width/height so the page does not jump while loading (layout shift). `srcset` = several image sizes; `<picture>` + `<source>` = several formats.

Tables, lists, block vs inline, semantic tags

Topic	Remember
Table	`table`, `tr` (row), `th` (heading cell), `td` (data cell); `caption`, `thead`, `tbody`, `tfoot`. `rowspan` = cell spans rows, `colspan` = cell spans columns.
Lists	`ol` ordered (type 1, A, a, I, i), `ul` unordered (disc, circle, square, none), `li` = item.
Block elements	Take 100% available width, start on a new line, can have width/height. Examples: `h1`, `p`, `div`, `ul`.
Inline elements	Take only as much space as content, stay in the line, cannot take width/height (unless changed to block/inline-block). Examples: `span`, `a`.
Semantic tags (HTML5)	`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`. They describe meaning (good for SEO). `div` and `span` are NON-semantic.
Layout	Use `div`/semantic tags + CSS (flex/position), not tables. Tables are for tabular data.

PART 2 - CSS (Box Model, Flexbox, Media Queries, Positioning)

2.1 CSS basics

CSS = Cascading Style Sheets. It controls how a page looks: layout, colour, font, borders, spacing, and responsiveness (mobile-friendly). Syntax: `selector { property: value; property2: value2; }`

Selector	Example	Selects
Tag	`p { color: red; }`	All `<p>` elements
ID	`#name { font-size: 20px; }`	The element with `id="name"` (unique on the page)
Class	`.alpha { font-family: arial; }`	All elements with `class="alpha"` (reusable)
Universal	`* { ... }`	Everything
Descendant	`div p { }`	Any `p` inside a `div`
Child	`form > div { }`	Only DIRECT children
Pseudo-class	`a:hover { }`	An element in a state

Three ways to add CSS

Way	How	Notes
Inline	`style="color:red"` attribute on the element	One element only; hard to maintain
Internal	`<style>` element, usually in `<head>`	One page only
External	`<link rel="stylesheet" href="style.css"/>` in head	Best: one file shared by many pages

Priority if rules clash: inline beats ID, ID beats class, class beats tag (specificity). If equal, the later rule wins (that is the 'cascading').

2.2 The CSS Box Model (MUST KNOW, includes calculations)

Every element is drawn as a rectangular box. From the inside to the outside:

Layer	What it is	Properties
1. Content	The text/image itself	`width`, `height`
2. Padding	Space between content and border (inside the border, shows background colour)	`padding`, `padding-top/right/bottom/left`
3. Border	The line around the padding	`border-width`, `border-style`, `border-color` (or `border: 2px solid red`)
4. Margin	Empty space OUTSIDE the border, between this and other elements (transparent)	`margin`, `margin-top/right/bottom/left`

- **Default** `box-sizing: content-box`**:** `width` and `height` apply to the CONTENT only. Padding and border are added on top.
- `box-sizing: border-box`**:** `width`/`height` include content + padding + border (border to border). Margin is still outside.
- Shorthand order is clockwise from the top: `padding: 10px 20px 30px 40px` = top, right, bottom, left. Two values `10px 20px` = top/bottom 10, left/right 20. One value = all four sides.
- `margin: 0 auto` centres a block element that has a width.
- Top/bottom margins, width and height do NOT work on inline elements. Use `display: inline-block` or `block`.

The calculations (practise these!)

Given: width 200px, padding 20px, border 5px, margin 10px	content-box (default)	border-box
Content width	200px	200 - 2x20 - 2x5 = 150px
Visible box (content + padding + border)	200 + 40 + 10 = **250px**	**200px**
Total space taken on page (add margins)	250 + 20 = **270px**	200 + 20 = **220px**

```
* { box-sizing: border-box; }   /* common reset: sizes behave as you expect */
div { width: 200px; padding: 20px; border: 5px solid black; margin: 10px; }
```

2.3 Positioning (MUST KNOW)

Value	Behaviour	Keeps its original space?
`static`	**Default.** Normal flow. `top/left/...` have no effect.	Yes
`relative`	Moved relative to its OWN normal position (using top/left/...). Others do not move to fill the gap.	**Yes**
`absolute`	Removed from normal flow. Placed relative to the **nearest positioned ancestor** (an ancestor with position other than static). If none, relative to the page.	**No**
`fixed`	Placed relative to the **viewport** (browser window). Stays in the same place when you scroll.	No
`sticky`	Acts like `relative` until a scroll point, then sticks like `fixed` until its parent scrolls out of view.	Yes (while normal)

```
.parent { position: relative; }                    /* makes parent the reference */
.child  { position: absolute; top: 0; right: 0; }  /* child sits in parent's top-right corner */
.header { position: sticky; top: 0; }              /* sticks at top while scrolling */
```

EXAM TRAP: To position an absolute child relative to its container, the container must be positioned (`relative`, `absolute` or `fixed`). Otherwise the child positions itself against the whole page.

2.4 Flexbox (MUST KNOW)

Flexbox lays out items in one direction (a row or a column). Turn it on with `display: flex` on the **container**. The container's direct children are **flex items**.

Axes

- **Main axis** = the direction items are laid out. **Cross axis** = perpendicular to it.
- `flex-direction: row` (default): main axis is horizontal (left to right), cross axis is vertical.
- `flex-direction: column`: main axis is vertical, cross axis is horizontal.
- Flex uses **start and end**, not left and right (because they flip with direction; `row-reverse` starts from the right).
- By default, items start at the start of the main axis, do NOT stretch along the main axis, but DO stretch across the cross axis.

Container properties

Property	Works on	Values / notes
`flex-direction`	Sets main axis	`row` (default), `column`, `row-reverse`, `column-reverse` (4 options)
`justify-content`	**Main axis** (spreads the free space)	`flex-start` (default), `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`
`align-items`	**Cross axis**	`stretch` (default if there is free space), `flex-start`, `flex-end`, `center`, `baseline`. Needs spare space in the cross axis to show any effect (e.g. container has a height).
`flex-wrap`	Whether items go to a new line	`nowrap` (default - items stay on one line and may overflow/shrink), `wrap`, `wrap-reverse`

Item properties

Property	Default	Meaning
`flex-grow`	`0`	How POSITIVE free space (spare room) is shared. 0 = do not grow. Bigger number = bigger share.
`flex-shrink`	`1`	How NEGATIVE free space (overflow) is taken out. 1 = items shrink to fit. 0 = never shrink.
`flex-basis`	`auto`	Starting size on the main axis before growing/shrinking. `auto` = use `width` if set, otherwise content size. `0` = ignore content and width. If both `width` and `flex-basis` are set, **basis wins**.
`flex`	`0 1 auto`	Shorthand: `flex: grow shrink basis`. `flex: 1` = grow 1, share equally.

Positive vs negative free space (diagram idea from slides)

- Container 500px, items total 300px -> **positive free space = 200px** (use `justify-content` or `flex-grow` to use it).
- Container 500px, items total 600px -> **negative free space = 100px overflow** (items shrink because `flex-shrink: 1`, or overflow if shrink is 0).
- To make items grow only up to a limit and then spread the rest, combine `flex-grow` with `max-width` and `justify-content`.

```
.container { display: flex; flex-direction: row; justify-content: space-between;
             align-items: center; flex-wrap: wrap; height: 300px; }
.item { flex: 1 1 100px; }    /* grow 1, shrink 1, basis 100px */
```

MEMORY TRICK: justify-content = MAIN axis. align-items = CROSS axis. Think: 'J' = Journey direction (main), 'A' = Across.

2.5 Media queries (responsive design)

Media queries apply CSS only when a condition is true, such as the screen width. They make one page work on phone, tablet and desktop.

```
/* applies when screen is 600px wide or LESS (phones) */
@media (max-width: 600px) {
    .container { flex-direction: column; }
    .menu { display: none; }
}

/* applies when screen is 768px wide or MORE (tablets/desktops) */
@media (min-width: 768px) {
    .container { flex-direction: row; }
}
```

- `max-width: X` = condition true when width is X **or smaller**. `min-width: X` = true when width is X **or larger**.
- Combine conditions with `and`: `@media (min-width: 600px) and (max-width: 900px)`.
- Add `<meta name="viewport" content="width=device-width, initial-scale=1">` in `<head>` so phones use their real width.
- Typical use with flexbox: row layout on desktop, column layout on phone.

2.6 Short extras from the CSS slides

Topic	Remember
Link states	`a:link`, `a:visited`, `a:hover`, `a:active`. Write them in this order (LoVe HAte).
Font/colour	`color` (text), `background-color`, `font-family`, `font-size`, `font-weight`, `font-style`, `height`, `width`.
Text (lower priority)	`text-align`, `text-indent`, `line-height`, `word-spacing`, `letter-spacing`. (Slide says you may skip these for exams.)
Background image	`background-image`, `background-repeat`, `background-position`, `background-attachment`.
Lists	`list-style-type` (none, disc, circle, square), `list-style-image`, `list-style-position` (inside/outside). Shorthand: `list-style: circle url("texture.jpg") inside;`

Topic	Remember
Navigation bar	Style a `ul`. `li` is `list-item` by default; change to `display: inline-block` (or use flex) to put items in one row.
Float	`float: left/right` takes a block out of normal flow; later content wraps around it. A parent that holds only floated children collapses (height 0). `clear: left/right/both` stops wrapping.
Div vs table for layout	Prefer `div` with flex/position: cleaner code, fewer bytes, tables are for data.

PART 3 - Expected questions with answers

3A. Short questions - HTML and forms

Q1. What is the purpose of the `<form>` tag? Name its two main attributes.

It collects user input and sends it to a server. `action` = where to send the data (URL). `method` = how to send it (GET or POST).

Q2. What is the default method of a form?

GET.

Q3. Differentiate GET and POST.

GET puts the data in the URL (visible, bookmarkable, limited size, used for getting/searching). POST puts the data in the request body (not in URL, no practical size limit, used for login/registration/saving/uploading). See table 1.11.

Q4. Why should a password form use POST and not GET?

GET shows the password in the address bar, history and logs (example from slide: `password=secret`). POST keeps it out of the URL. HTTPS is still needed for real security.

Q5. What happens if an input has no `name` attribute?

Its value is not sent to the server.

Q6. What is the use of the `for` attribute of `<label>`?

It links the label to the input whose `id` matches. Clicking the label focuses/selects that input, and screen readers can read it properly.

Q7. How do you make a group of radio buttons where only one can be chosen?

Give all radio buttons the same `name`.

Q8. Difference between radio and checkbox?

Radio: select exactly one option of a group. Checkbox: select zero, one or many options. Radios share a name to form a group; checkboxes can share a name and send repeated parameters.

Q9. What is sent if radio/checkbox has no `value` attribute?

`on` (e.g. `gender=on&course=on&course=on`). So always give a `value`.

Q10. Are unchecked checkboxes sent to the server?

No. Only checked ones are sent.

Q11. How do you create a drop-down list? How do you pre-select an item?

`<select name="city">` with `<option>` items. Add `selected="selected"` to the option that should be pre-selected.

Q12. What is sent for `<option value="1">Lahore</option>` if the user picks Lahore?

`city=1` (the `value`). Without `value`, the text `Lahore` is sent.

Q13. How do you create a multi-line text box and set its size?

`<textarea name="comments" rows="6" cols="30"></textarea>`. `rows` = lines, `cols` = characters wide.

Q14. What is a hidden input field? Give a use. Is it secure?

An input of `type="hidden"` that is not displayed but is sent with the form, e.g. `<input type="hidden" name="id" value="23">` to send a record id. It is not secure; users can see and change it.

Q15. Name the button types and what each does.

`submit` sends the form; `reset` restores the starting values; `button` does nothing unless JavaScript is attached.

Q16. How can you tell which of several submit buttons was clicked?

Give them a `name` (and `value`). Only the clicked button's name=value is sent, e.g. `action=Update` or `action=Delete`.

Q17. Name HTML5 input types and the validation they give.

`email` (must look like an email, needs @), `url` (must be a valid URL), `date` (date picker). Also `required` stops empty submission.

Q18. What is the difference between `value` and `placeholder`?

`value` is real starting text and is sent. `placeholder` is only a grey hint that disappears when you type; it is not a value.

Q19. What is a UX tip for update (edit) forms?

Always show the current value in the fields. Only exception: password.

Q20. Write the URL produced when the user enters name Ali in a GET form with action `register.php`, and clicks submit. (field name `first-name`, email `a@b.com`, field name `email`)

`register.php?first-name=Ali&email=a@b.com`

Q21. What are `<meta charset="UTF-8">` and `dir="rtl"` used for?

`charset` sets the character encoding so languages like Urdu display correctly. `dir="rtl"` makes text flow right-to-left.

Q22. What are Open Graph meta tags?

Tags like `og:url`, `og:title`, `og:description`, `og:image` that tell social sites which title, description and image to show when a page is shared.

Q23. Block vs inline element?

Block: full width, starts a new line, can set width/height (`div`, `p`, `h1`). Inline: only content width, stays in the line, no width/height (`span`, `a`).

Q24. Name five HTML5 semantic tags. Why use them?

`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`. They show the meaning of content, which helps SEO and accessibility; `div`/`span` have no meaning.

3B. Short questions - CSS

Q25. What is CSS? Name the three ways to add it.

Cascading Style Sheets, used to style pages (layout, colour, fonts, responsiveness). Inline (`style` attribute), internal (`<style>` in head), external (`<link rel="stylesheet">`).

Q26. Explain tag, id and class selectors with examples.

Tag `p {}` selects all paragraphs; id `#name {}` selects the one element with that id; class `.alpha {}` selects all elements having that class.

Q27. Explain the CSS box model.

Every element is a rectangle made of content, then padding, then border, then margin (inside to outside). Width/height normally set the content area; padding and border add to the size; margin is outside space.

Q28. What does `box-sizing: border-box` do?

It makes width/height include content + padding + border, so the box stays exactly the width you set. (Default is `content-box`.)

Q29. Calculate the total width: width 300px, padding 20px, border 5px, margin 10px, content-box.

Visible box = 300 + 2x20 + 2x5 = 350px. Space on page with margins = 350 + 2x10 = 370px. (With border-box: visible = 300px; page space = 320px.)

Q30. Difference between padding and margin?

Padding is space inside the border (between content and border, shows the background). Margin is space outside the border (between elements, transparent).

Q31. Explain the 5 position values.

static (default, normal flow); relative (offset from own normal place, keeps its space); absolute (out of flow, relative to nearest positioned ancestor, no space kept); fixed (relative to viewport, stays on scroll); sticky (relative until a scroll point, then fixed).

Q32. Relative vs absolute positioning?

Relative moves from its own original position and still occupies its original space. Absolute is removed from the flow, takes no space, and is placed against the nearest positioned ancestor (or the page).

Q33. How do you place an absolute child in the corner of its parent?

Set the parent to `position: relative` and the child to `position: absolute; top: 0; right: 0;`.

Q34. What is flexbox? How do you create a flex container?

A one-dimensional layout system for rows or columns. Add `display: flex` to the container; its direct children become flex items.

Q35. What are the main axis and cross axis?

Main axis = direction of the items (`row` -> horizontal, `column` -> vertical). Cross axis = the perpendicular direction.

Q36. List the values of `flex-direction`.

`row` (default), `column`, `row-reverse`, `column-reverse`.

Q37. Difference between `justify-content` and `align-items`?

`justify-content` distributes free space along the main axis; `align-items` aligns items along the cross axis.

Q38. Name values of `justify-content`.

`flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`.

Q39. What does `flex-wrap` do? What is its default?

It decides whether items can move to a new line when there is not enough room. Default `nowrap` (one line; items shrink or overflow). `wrap` allows new lines.

Q40. Explain `flex-grow`, `flex-shrink`, `flex-basis` with defaults.

flex-grow (0): share of positive free space; flex-shrink (1): how much an item shrinks when space is negative; flex-basis (auto): starting size on the main axis, `auto` = width or content size.

Q41. What is positive and negative free space in flexbox?

Positive: container is bigger than the items together (spare room, can be shared by `flex-grow`/`justify-content`). Negative: items together are bigger than the container (overflow, taken back by `flex-shrink`).

Q42. Why does `align-items: center` sometimes do nothing?

Because there is no spare space in the cross axis (for a row, the container has no extra height). Give the container a height.

Q43. What is a media query? Write one for screens up to 600px wide.

CSS that applies only if a condition is true. `@media (max-width: 600px) { ... }`

Q44. Difference between `min-width` and `max-width` in media queries?

`min-width: X` applies when width is X or more (bigger screens). `max-width: X` applies when width is X or less (smaller screens).

Q45. Why use div and flex/position for layout instead of tables?

Tables are meant for tabular data; using them for layout makes complex code, more markup bytes and a code smell. Div + flex/position is cleaner and more flexible.

Q46. How do you style link states? In what order?

`a:link`, `a:visited`, `a:hover`, `a:active` (this order).

3C. 'Differentiate between...' quick tables

Pair	Difference in one line each
GET vs POST	GET: data in URL, bookmarkable, limited, for fetching. POST: data in body, hidden from URL, large, for submitting/changing.
Radio vs Checkbox	Radio: one choice per group (same name). Checkbox: any number of choices.
submit vs reset vs button	submit sends the form; reset restores initial values; button needs JavaScript.
id vs class	id: unique, one element, `#`. class: reusable, many elements, `.`.
Inline vs Internal vs External CSS	Inline: style attribute, one element. Internal: style tag, one page. External: .css file, many pages.
Padding vs Margin	Padding: inside border. Margin: outside border.
content-box vs border-box	content-box: width = content only. border-box: width includes padding and border.
Relative vs Absolute	Relative keeps space, moves from own place. Absolute leaves flow, uses nearest positioned ancestor.
Absolute vs Fixed	Absolute: against positioned ancestor, scrolls with page. Fixed: against viewport, stays on scroll.
Fixed vs Sticky	Fixed always stays. Sticky is normal until a scroll threshold, then sticks.
justify-content vs align-items	Main axis vs cross axis.
flex-grow vs flex-shrink	grow uses spare space (default 0). shrink removes overflow (default 1).
Block vs Inline	Block: new line, full width, sizeable. Inline: in the line, content width, not sizeable.
placeholder vs value	Hint text (not sent) vs real default text (sent).
min-width vs max-width (media)	Bigger-or-equal screens vs smaller-or-equal screens.

3D. Predict the output / what happens? (very likely type)

P1. User types nothing in first name, selects Male, ticks OOP and DB. What URL?

```
<input type="text" name="first-name"/>
<input type="radio" name="gender" value="Male"/> Male
<input type="radio" name="gender" value="Female"/> Female
<input type="checkbox" name="course" value="OOP"/> OOP
<input type="checkbox" name="course" value="DB"/>  DB
```

Answer: `register.php?first-name=&gender=Male&course=OOP&course=DB` (empty field still appears; both ticked boxes appear). If the radio/checkbox had NO value attributes it would be `...&gender=on&course=on&course=on`.

P2. User types Ali, picks Lahore, writes hello in the comments. What URL?

```
<select name="city">
  <option>Select</option>
  <option value="1">Lahore</option>
  <option value="2">Karachi</option>
</select>
<textarea name="comments"></textarea>
```

Answer: `register.php?first-name=Ali&city=1&comments=hello` (select sends the option's `value`, 1).

P3. User types Ali and clicks the Update button. What URL?

```
<input type="text" name="first-name"/>
<input type="submit" name="action" value="Update"/>
<input type="submit" name="action" value="Delete"/>
```

Answer: `register.php?first-name=Ali&action=Update` (only the clicked button is sent).

P4. Same form but the buttons have no name attribute. What is sent for buttons?

```
<input type="submit" value="Update"/>
```

Answer: Nothing from the button. URL would only be `register.php?first-name=Ali`.

P5. A hidden field exists. What URL after typing Ali?

```
<input type="hidden" name="city" value="Lahore"/>
<input type="hidden" name="id" value="23"/>
<input type="text" name="first-name"/>
```

Answer: `register.php?city=Lahore&id=23&first-name=Ali` (follows the order in the HTML).

P6. Container 500px wide, three items 100px wide each. What happens with `justify-content: space-between`?

```
.container { display: flex; width: 500px; justify-content: space-between; }
.item { width: 100px; }
```

Answer: Free space = 500 - 300 = 200px. First item at the left edge, last at the right edge, 100px gap between each pair.

P7. Container 500px, three items 200px each, default flex settings. What happens?

```
.container { display: flex; width: 500px; }
.item { width: 200px; }
```

Answer: Items total 600px = 100px negative free space. `flex-shrink` defaults to 1, so each shrinks equally to about 166.7px and all fit on one line. With `flex-shrink: 0` they would overflow by 100px. With `flex-wrap: wrap`, two items fit on line 1 and the third wraps to line 2.

P8. Container 600px; A, B, C each have flex-basis 100px. A: grow 1, B: grow 2, C: grow 0. Final widths?

```
.A { flex: 1 1 100px; }
.B { flex: 2 1 100px; }
.C { flex: 0 1 100px; }
```

Answer: Free space = 600 - 300 = 300px. Shared in ratio 1:2 (C gets none): A gets 100, B gets 200. Final: A = 200px, B = 300px, C = 100px (total 600).

P9. Where does the child appear?

```
.parent { position: relative; }
.child  { position: absolute; top: 0; right: 0; }
```

Answer: In the top-right corner of the parent. If the parent were `static`, the child would go to the top-right of the page.

P10. Total space used by the box?

```
div { width: 200px; padding: 20px; border: 10px solid; margin: 15px; }
```

Answer: content-box: visible = 200 + 40 + 20 = 260px; with margins 260 + 30 = **290px**. With `box-sizing: border-box`: visible = 200px (content = 140px); with margins = **230px**.

3E. Find the mistake

B1. The user can select both Male and Female. Why?

```
<input type="radio" name="g1" value="Male"/> Male
<input type="radio" name="g2" value="Female"/> Female
```

Answer: The radios have different names. Same `name` is needed to group them.

B2. Clicking the label does not focus the textbox. Why?

```
<label for="fname">First Name</label>
<input type="text" name="fname"/>
```

Answer: The input has no `id="fname"`. `for` matches `id`, not `name`.

B3. The server never receives the email. Why?

```
<input type="email" placeholder="Email"/>
```

Answer: There is no `name` attribute.

B4. The server cannot tell if the user chose Web or OS. Why?

```
<input type="checkbox" name="course" value="Web"/> Web
<input type="checkbox" name="course" value="Web"/> OS
```

Answer: Both have the same `value`. Each option needs a unique value (OS should have `value="OS"`).

B5. What is wrong with this login form?

```
<form action="login.php" method="GET">
  <input type="password" name="password"/>
</form>
```

Answer: GET shows the password in the URL. Use `method="POST"` (and HTTPS).

B6. The badge is not in the corner of the card, it goes to the page corner. Why?

```
.card  { width: 300px; }
.badge { position: absolute; top: 0; right: 0; }
```

Answer: `.card` is not positioned. Add `position: relative` to `.card`.

B7. align-items: center has no visible effect. Why?

```
.row { display: flex; align-items: center; }
```

Answer: The container has no extra height, so there is no free space in the cross axis. Add a `height`.

B8. I want a column layout on phones but it is applied on big screens. Fix it.

```
@media (min-width: 600px) { .box { flex-direction: column; } }
```

Answer: Use `max-width: 600px` for small screens. `min-width` applies to wider screens.

3F. True or False

#	Statement	Answer
1	POST encrypts the data by default.	False. It only keeps data out of the URL; HTTPS encrypts.
2	A GET request can be bookmarked.	True.
3	The default form method is POST.	False. It is GET.
4	Radio buttons with different names behave as one group.	False. Same name makes a group.
5	The `for` of a label matches the input's `id`.	True.
6	A hidden field is safe because users cannot see it.	False. They can view and edit it.
7	Unchecked checkboxes are sent as empty values.	False. They are not sent at all.
8	The reset button submits the form.	False. It restores initial values.
9	`required` replaces server-side validation.	False. Browser checks can be bypassed.
10	A textarea uses a `value` attribute for its text.	False. The text goes between the tags.
11	Margin is inside the border.	False. Padding is inside; margin is outside.
12	Default `box-sizing` is `content-box`.	True.
13	A relatively positioned element keeps its original space.	True.
14	An absolutely positioned element keeps its original space.	False.
15	A fixed element is positioned relative to the viewport.	True.
16	`justify-content` works on the cross axis.	False. Main axis.
17	`flex-wrap` default is `nowrap`.	True.
18	`flex-grow` default is 1.	False. It is 0 (and `flex-shrink` is 1).
19	Width and height apply to inline elements.	False.
20	Tables are the best tool for page layout.	False. Use div + flex/position.

PART 4 - One-page cheat sheet (read this last, 10 minutes before the quiz)

FORMS	
Form attrs	`action` = where, `method` = how (default GET)
Sent only if	field has `name` (checkbox ticked, radio chosen, button clicked)
Radio	same `name` = one group; no `value` -> `on`
Checkbox	repeats: `course=OOP&course=DB`; unchecked = not sent
Select	`selected`; no option value -> text sent
Textarea	`rows`, `cols`; text between tags
Buttons	submit sends, reset restores, button = JS only; name needed to send button value
Hidden	sent but invisible; NOT secure
Label	`for` = input `id`
HTML5	email, url, date, `required`, `placeholder` (hint only)
GET	URL, visible, bookmarkable, small, for getting
POST	body, not in URL, large, uploads, for login/saving; needs HTTPS to be safe

CSS	
Box model	content > padding > border > margin. Default content-box; border-box includes padding+border.
Math	visible = width + 2*padding + 2*border (content-box). Page space = visible + 2*margin.
Position	static default | relative keeps space | absolute -> nearest positioned ancestor, no space | fixed -> viewport | sticky -> relative then fixed
Flex on	`display: flex` on container; children = items
Axes	row: main horizontal; column: main vertical
justify-content	MAIN axis: flex-start, flex-end, center, space-between, space-around, space-evenly
align-items	CROSS axis: stretch(default), flex-start, flex-end, center, baseline
flex-wrap	nowrap (default), wrap, wrap-reverse
Item props	grow 0, shrink 1, basis auto; `flex: g s b`
Free space	positive = spare room (grow); negative = overflow (shrink)
Media query	`@media (max-width: 600px) { }` small screens; `min-width` bigger screens
Link order	link, visited, hover, active

LAST WORD: Good luck! If you can explain GET vs POST, write the URL from a form, and calculate box width, you will already cover most of the quiz.